

PROJECT DOCUMENTATION

ScribeTube

Premium AI-Powered YouTube Video Summarizer

ScribeTube is a professional-grade web application built using Python, Flask, and the Google Gemini API. It extracts transcripts from YouTube videos using the YouTube Transcript API and generates highly structured summaries including Executive Summaries, Key Insights, Core Takeaways, and tailored visual concept breakdowns (processes, timelines, comparisons, or metrics).

Author / Role:	Internship Project
Date Generated:	July 11, 2026
Version:	1.0.0
Development Stack:	Python 3.14, Flask, Google Gemini SDK, Vanilla HTML/CSS/JS

1. Executive Summary & Overview

In the modern digital landscape, video has become a primary medium for education, programming tutorials, and documentation. However, consuming long-form content is time-consuming. **ScribeTube** resolves this by providing instant, structured summaries of YouTube videos directly from their transcriptions. By utilizing state-of-the-art Generative AI models from Google (Gemini), ScribeTube generates professional-grade, segmented notes that highlight the speaker's main thesis, key arguments, and specific takeaway messages.

Key Value Propositions:

- **Time Efficiency:** Turns a 30-minute video lecture into a structured 1-minute read.
- **Visual Representation:** Automatically chooses between timelines, workflows, comparisons, or metric scorecards based on video content.
- **State Persistence:** Keeps a local history log and favorites dashboard for immediate recall without repeating API queries.

Core User Features:

- **Executive Summary:** Cohesive, multi-paragraph synopsis explaining context, main topics, and speakers' core thesis.
- **Key Insights:** Actionable bulleted takeaways with bold titles highlighting crucial arguments and details.
- **Key Takeaway:** A concise, single-sentence concluding statement summarizing the ultimate value of the video.
- **Concept Visual Breakdown:** Interactive UI widgets dynamically selected based on content structure (chronological timeline, step-by-step setup guides, comparative lists, or metrics).
- **Theme Switcher:** Sleek theme toggle supporting light and dark modes with customized neon ambient glows.
- **Action Utilities:** Buttons to copy individual sections or the full summary to the clipboard, download summary text files, and manage saved histories.

2. System Architecture & Tech Stack

ScribeTube is engineered as a clean, single-page application (SPA) client combined with a lightweight Python Flask backend. The server handles heavy API communications and sanitization, while the browser governs interface layouts, search filtering, and state caching.

Component	Technologies Used	Role in Application
Backend Server	Python 3.14, Flask 3.1.3	Exposes API routes, handles validation, manages transcript fetching, and coordinates Gemini API queries.
AI Inference	google-generativeai 0.8.6 (Gemini SDK)	Analyzes transcripts and produces highly structured, validated JSON data via structured prompts.
Transcript Fetcher	youtube-transcript-api 1.2.4	Extracts captions and subtitles directly from YouTube servers by video ID.
Frontend UI	HTML5, Vanilla CSS3, ES6+ Javascript	Renders glassmorphic dashboard views, handles user tabs, toggles themes, manages clipboard copy, and caches data.
Persistence Cache	Web LocalStorage API	Stores dynamic arrays for recent summary history and starred favorites directly inside user's browser storage.

System Flowchart Sequence:

1. User Input	User pastes YouTube URL into input field and submits form. JS validates format.
2. Metadata & API Call	JS triggers loader animations and sends POST payload containing URL to /api/summarize.
3. Video Extraction	Flask extracts the 11-character video ID and calls YouTube oEmbed endpoint to retrieve title/ch
4. Transcript Pull	Flask fetches the subtitle transcript. First looks for English ('en'), then falls back to other languag
5. AI Summarization	Flask constructs a structured JSON prompt and calls Gemini API (using fallback list).
6. Response Sanitization	Flask receives AI response, cleans markdown braces, parses JSON, adds metadata, and return
7. UI Rendering	JS pushes summary to LocalStorage, updates sidebar lists, formats key points, and renders vis

3. Backend Implementation & API Design

The Python backend serves as the gateway to transcript scrapers and AI endpoints. It is implemented in **app.py** and coordinates configuration loading, exception translations, and AI parsing rules.

Key Core Functions:

extract_video_id(url):

Robust extraction utility matching various formats of YouTube links (regular, mobile, shortenedyoutu.be, embeds, live, or raw IDs). It uses a comprehensive regular expression:

```
pattern = r'(?:(https?:\/\/)?(?:www\.)?(?:m\.)?(?:music\.)?(?:youtube\.com\/(?:[^\n\s]+\/\S+\/|(?:(v|e(?:mbed)?|live)\.\/|S*?[?&]v=)|youtu\.be\/)([a-zA-Z0-9_-]{11}))'
```

fetch_youtube_transcript(video_id):

Responsible for retrieving video transcripts via the youtube-transcript-api. To increase reliability, it prioritizes English tracks but gracefully falls back to the first available language track. It captures and translates exceptions into friendly consumer errors:

- **TranscriptsDisabled / Subtitles disabled:** Subtitles/transcripts are disabled or unavailable for this video.
- **VideoUnavailable:** This YouTube video is unavailable, private, or restricted.
- **InvalidVideoId:** The YouTube video ID extracted from the URL is invalid.
- **IpBlocked / Too Many Requests:** YouTube transcript retrieval was rate-limited or blocked. Please try again later.

fetch_video_details(video_id):

Retrieves video title, author/channel name, and author URL by making a GET request to YouTube's official oEmbed endpoint. This ensures that even without heavy API keys, the user gets visual context of the video they are summarizing.

Gemini API Structured Schema Prompting:

To get consistent, structured answers, the prompt forces Gemini to respond strictly in JSON matching an explicit schema. Furthermore, to handle hot-swappable setups or potential server availability issues, the backend uses a sequential model fallback array. It attempts generation in this order:

gemini-3.1-flash-lite, gemini-flash-latest, gemini-2.5-flash, gemini-2.0-flash, gemini-1.5-flash

clean_json_string(raw_text):

Often, LLM models return markdown blocks (e.g. ```json ... ```) or trailing braces. This utility performs sanitization by cropping markdown tags, trimming spaces, extracting the first/last braces, and running a fast-path parser. If that fails, a slow-path balancer scans brace nesting counts to construct a valid parseable JSON document.

4. Frontend Implementation & UI Design

The frontend is built using standard web technologies. It is fully responsive and leverages dark/light styling rules defined in **style.css**, with DOM updates and data routing controlled by **app.js**.

UI Layout & Styling System:

ScribeTube's stylesheet establishes a dark-by-default palette utilizing glassmorphism visual aesthetics. Key CSS modules specify:

- **Glass Cards:** Cards styled with semi-transparent backdrops, blur filters (`backdrop-filter: blur(16px)`), and border highlight reflections.
- **Glow Background:** Floating ambient color gradients that animate in the background.
- **Responsive Drawer:** An off-canvas navigation sidebar that acts as a slider on mobile viewports and pins to the side on desktops.
- **Skeleton Loaders:** Pulse animations representing cards and lines to keep the UI active while transcripts are parsed.
- **Theme Toggles:** Transitions toggling between a purple-tinted dark mode and a high-contrast warm light mode.

Interactive JavaScript Modules (`app.js`):

Client side scripting coordinates API responses, transitions, caches, and templates:

State Caching & Synchronization:

Loads recent queries and starred favorites from `localStorage` (keys: `scribetube_history` and `scribetube_favorites`). It synchronizes changes so that pinning a video automatically updates the active button label, sidebar quick list, history tab list, and starred favorites tab grid.

Loading Rotator:

Since AI processing might take several seconds, JavaScript cycles through a sequence of informative loader status lines (e.g., 'Extracting video transcript...', 'Analyzing transcript core content...', 'Structuring executive summaries...') every 3 seconds to reassure the user of active backend calculations.

Concept Visual Breakdown Renderer:

Based on the `visual_elements.type` field output from Gemini, Javascript injects customized layout cards:

- **timeline:** Injects timeline nodes containing bullet times and detailed description rows.
- **process:** Generates a sequential step-by-step wizard highlighting sequential guide cards.
- **comparison:** Dynamically builds comparisons in HTML tables with custom columns.

- **key_metrics:** Populates metrics inside high-impact scorecards.

5. Deployment & Testing Strategy

Local Setup & Execution Guide:

Follow these steps to deploy ScribeTube in a local development environment:

- Clone/navigate to the project directory: `c:\Users\divya\Desktop\Intern\Youtube Summarizer`.
- Create a virtual environment: `python -m venv venv` and activate it.
- Install all required packages: `python -m pip install -r requirements.txt`.
- Copy the environment configuration template: `copy .env.example .env`.
- Open `.env` in an editor and enter your Gemini API Key.
- Start the Flask web server: `python app.py`.
- Open your browser and navigate to the application dashboard: `http://localhost:5000`.

Core Environment Variables (`.env`):

Variable Key	Default Value / Format	Description
GEMINI_API_KEY	<i>your_api_key_string</i>	Crucial Google AI Studio API key used to invoke Gemini models. Summary fails if missing.
FLASK_ENV	development	Toggles Flask configuration mode.
FLASK_DEBUG	True	Enables hot-reloading and detailed debug trace logs.
PORT	5000	Server port parameter governing host listening socket.

Unit Testing & Mocking Strategy:

The project is covered by automated unit tests in `test_app.py`, which verifies the reliability of the application's utilities. It uses `unittest.mock.patch` to mock external APIs, shielding development from actual API quotas. You can run the tests with the following command:

```
python -m unittest test_app.py
```


Key Tests Implemented:

- **test_extract_video_id:** Tests robust extraction rules matching 7 different valid YouTube formats while rejecting invalid search terms.
- **test_serve_index:** Validates that the base application URL returns a HTTP 200 code serving index files.
- **test_config_check:** Ensures endpoint /api/config-check successfully reports configurations status.
- **test_summarize_missing_url / test_summarize_invalid_url:** Validates error handlers and bad request payloads returning HTTP 400 responses.
- **test_summarize_success_mock:** Mocks out both YouTube transcript scraping and Gemini inference engines to validate end-to-end response layouts, formatting, and video ID bindings.